

LAB 01

Dance imitation

Dance Imitation: Timed Choreography as Feedback Control

Environment design · PPO · Honest verification

BUILD → TRAIN → VERIFY → EXPLAIN



What you will build — and what the evidence says

- 01 Use case: Teach how PPO turns a time-indexed joint clip into a balanced 50 Hz feedback controller, then prove motion with deterministic ONNX tracking and null controls.
- 02 By the end: describe the scene, map observations to actions, explain the reward, trace one PPO update, run a bounded experiment, and defend an evaluation claim.
- 03 The claim covers one eight-second excerpt, not the complete 60.923-second source clip or arbitrary choreography.
- 04 Scope: local MuJoCo simulation. Do not deploy these classroom artifacts to a physical robot; hardware transfer requires a separate validated process.

Materials and prerequisites · 1/2

- 01 Apple Silicon Mac with the installed RLX environment, MLX, MuJoCo, ONNX, ONNX Runtime, Pillow, ImageIO, and FFmpeg.
- 02 Source choreography: `dance-clip/bachata_microduck_v2.clip.json`; the selected evidence uses its first unscaled eight seconds.
- 03 Prepared 400-pose reference: `rlx/artifacts/dance-e2e-20260907/reference.clip.json`.
- 04 Training and audit tools: `ppo_microduck_studio.py`, `dance_e2e.py`, `audit_dance.py`, and the Studio HTTP driver.

Materials and prerequisites · 2/2

05

Saved checkpoint, normalization sidecar, deterministic ONNX actor, evaluation JSON, MP4, contact sheet, and JSONL telemetry.

Environment and policy contract · 1/2

- 01 Nominal local MuJoCo with XML position actuators; physics dt 0.005 s, decimation 4, and control dt 0.02 s or 50 Hz.
- 02 Observation contract is float32 batch x 61: gyro 0:3, gravity 3:6, joint position 6:20, lagged velocity 20:34, last action 34:48, commands 48:61.
- 03 Dance keeps the shared contract by replacing body-command indices 59 and 60 with $\sin(\text{phase})$ and $\cos(\text{phase})$; twist commands are zero.
- 04 Action contract is float32 batch x 14 in deployment joint order; raw actor means are stored, applied offsets clip to $[-4, 4]$ rad, then add DEFAULT_POSE.

Environment and policy contract · 2/2

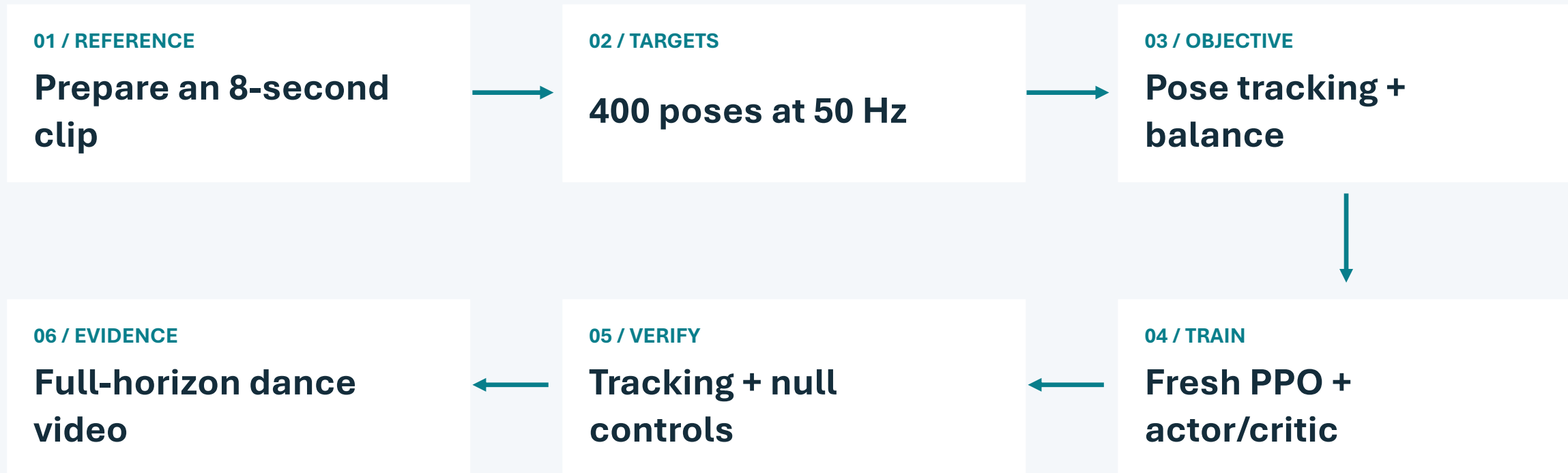
05

The selected run disables domain randomization, observation noise, action delay, and random yaw; normal reset perturbations still remain.

06

The clip loader uses 14 joint targets and rootPitch; authored root yaw, roll, position, and contact annotations are not imitation targets.

How this case is built: one evidence loop



Failure is data: change one cause, retrain, and repeat the same evaluation gates.

Build the case step by step · 1/2

- 01 Hash the 60.923 s source clip and prepare a looping, unscaled eight-second excerpt sampled into 400 control-rate poses.
- 02 Create the Dance recipe with per-joint Gaussian pose precision sigma 0.2, 16 environments, 128-step rollouts, and fresh actor-critic weights.
- 03 Train stochastic PPO while logging normalized collection reward, raw completed-episode return, and actual minibatch optimizer metrics separately.
- 04 Save checkpoints at rollout boundaries, including observation and reward normalization statistics and exact recipe metadata.

Build the case step by step · 2/2

05

Export the deterministic actor mean to ONNX with observation normalization embedded; the critic and action sampling are not exported.

06

Run strict complete-episode evaluation, then render the exported policy and independently compare trained, zero, untrained, and frozen-phase controls.

PPO: improve the actor without trusting a huge step

$$\text{ratio} = \pi_{\text{new}}(a \mid o) / \pi_{\text{old}}(a \mid o)$$

Maximize $\text{mean}[\min(\text{ratio} \times A, \text{clip}(\text{ratio}, 1 - \epsilon, 1 + \epsilon) \times A)]$

Actor: chooses joint actions. Critic: estimates future discounted reward.

A = advantage. ϵ = clipping range. Old log probabilities stay fixed for each collected batch.

Follow one rollout through the learner

- 01 Collect observations, sampled actions, old log probabilities, values, rewards and episode-boundary flags using the current stochastic actor.
- 02 Compute TD residuals and generalized advantage estimates (GAE). A true terminal has zero continuation value; a timeout may bootstrap the saved final observation.
- 03 Cut advantage traces at reset boundaries. Never bootstrap a timeout from the first observation of the next episode.
- 04 Reuse this batch for PPO minibatch updates, then collect fresh on-policy data. Export the deterministic actor and its observation transform for evaluation.

Paper exercise: count samples and clip the objective

- 01 Illustrative arithmetic, NOT this run: $16 \text{ environments} \times 128 \text{ steps} = 2,048$ fresh transitions per rollout. Four epochs reuse those transitions; they do not create 8,192 new samples.
- 02 With $\epsilon = 0.2$, ratio = 1.4 and $A = +2$: $\min(2.8, 2.4) = 2.4$. The minimized policy term is -2.4 before averaging and additional loss terms.
- 03 With the same ratio and $A = -2$: $\min(-2.8, -2.4) = -2.8$. Explain why clipping is asymmetric once the sign of the advantage changes.
- 04 Run from repo root: `python3 docs/microduck-training-student-book/learning_lab.py --gae`. This is a deterministic arithmetic lab, not new policy training.

Reward design: what the agent is paid to do · 1/2

- 01 pose_match 40: mean over joints of $\exp(-((q - q_{\text{ref}})/0.2)^2)$, replacing the broader inherited pose kernel.
- 02 rotation_match 4: rewards agreement with the clip's reference root pitch.
- 03 on_feet 5: rewards supported, upright physical execution while following the choreography.
- 04 no_slip 0.5 and no_spin 0.3: non-positive penalties for foot slip and excessive spin.

Reward design: what the agent is paid to do · 2/2

05

gentle_head 1, soft_landings 0.75, no_limit_parking 1, and save_energy 0.0005 are active penalty terms.

06

travel 0 disables forward-travel reward; stick_it 5 is present but inactive because the selected reference loops.

Recorded training recipe · 1/2

- 01 Actual selected run: seed 7, fresh PPO, 4,001,792 transitions, 16 fork environments, 128 steps per environment, and 2,048 transitions per rollout.
- 02 PPO uses 4 minibatches x 4 epochs: 512 rows per minibatch, 16 optimizer steps per rollout, 1,954 rollouts, and 31,264 optimizer steps.
- 03 Learning rate 0.0001; gamma 0.99; GAE lambda 0.95; clip 0.2; value coefficient 1; entropy coefficient 0; max gradient norm 0.5.
- 04 Initial action standard deviation is 0.1; actor and critic are separate 61-512-256-128 networks with ELU activations.

Recorded training recipe · 2/2

05

Observation and discounted-return reward normalization are enabled; observation statistics are frozen into the exported ONNX graph.

06

Update log fields are phase, steps, seconds, optimizer_steps, mean_loss, policy_loss, value_loss, entropy, approximate_kl, clip_fraction, explained_variance, env_steps.

Hands-on commands: inspect, smoke, train · 1/3

01

```
From .:  
./scripts/train-dance.sh --dry-run  
Inspect the current bundled Dance pipeline without training.
```

02

```
From .:  
./scripts/train-dance.sh  
Run the current independent Full Dance pipeline with a fresh timestamped output.
```

Hands-on commands: inspect, smoke, train · 2/3

03

```
From .:  
rlx/.venv-microduck/bin/python rlx/scripts/dance_e2e.py --source dance-  
clip/bachata_microduck_v2.clip.json --output rlx/artifacts/dance-reproduction-YYYYMMDD-  
HHMMSS --seconds 8 --train-steps 4001792 --run-name dance-e2e-reproduction-YYYYMMDD-HHMMSS  
Prepare the exact historical eight-second Bachata reference and recipe; replace the  
timestamp token consistently.
```

04

```
From .:  
rlx/.venv-microduck/bin/python rlx/scripts/audit_dance.py --run  
rlx/runs/studio/dance/dance-e2e-reproduction-YYYYMMDD-HHMMSS --recipe rlx/artifacts/dance-  
reproduction-YYYYMMDD-HHMMSS/recipe.json --output rlx/artifacts/dance-reproduction-  
YYYYMMDD-HHMMSS/audit --seeds 101 102 103 104 105 --history-stride 2 --render  
Audit a completed exact-recipe reproduction over held-out seeds without further PPO  
training.
```

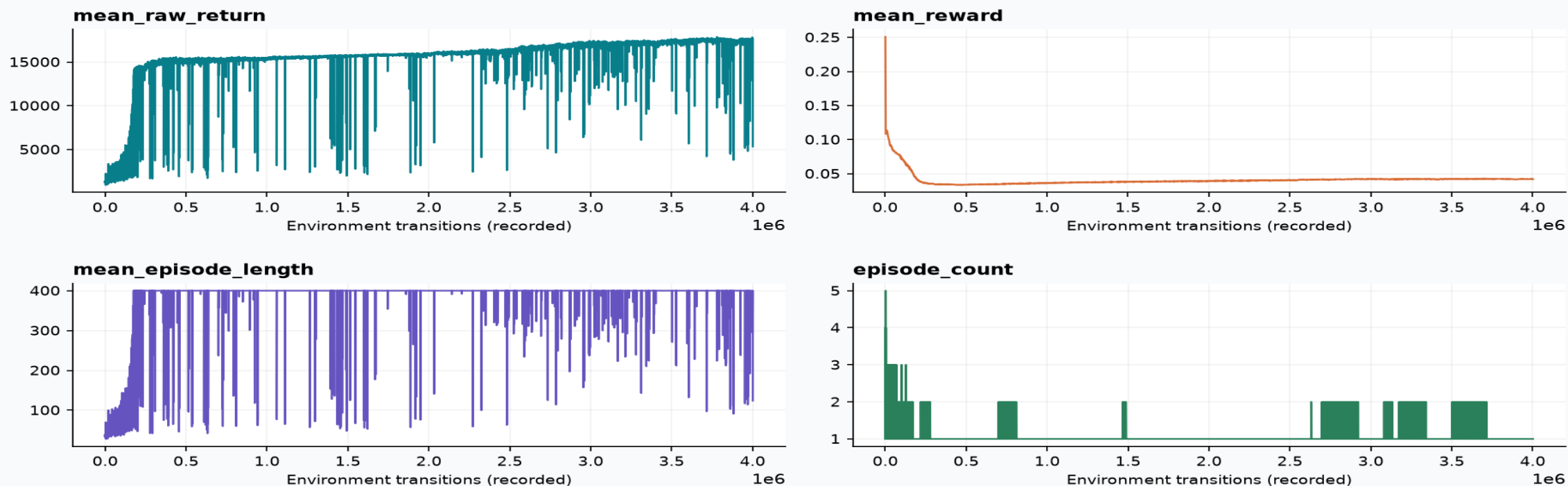
Hands-on commands: inspect, smoke, train • 3/3

05

```
From .:  
rlx/.venv-microduck/bin/python -m pytest rlx/tests/test_dance_evaluation.py  
rlx/tests/test_dance_pipeline.py rlx/tests/test_microduck_dance.py  
Run focused Dance contract and evaluation tests.
```

Reward: does the logged signal match the task?

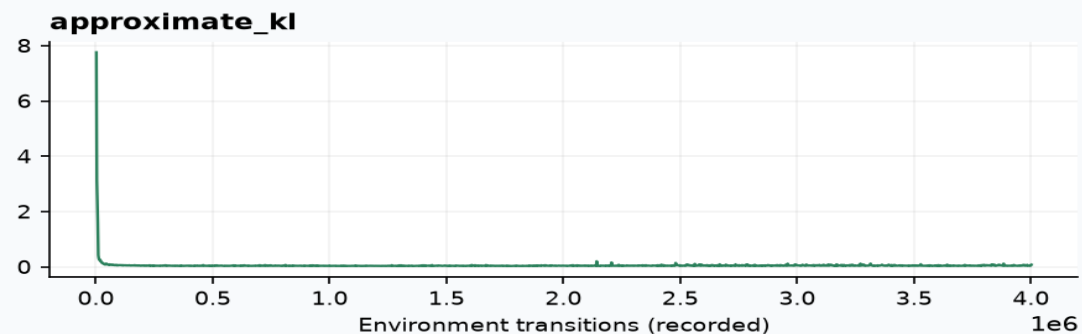
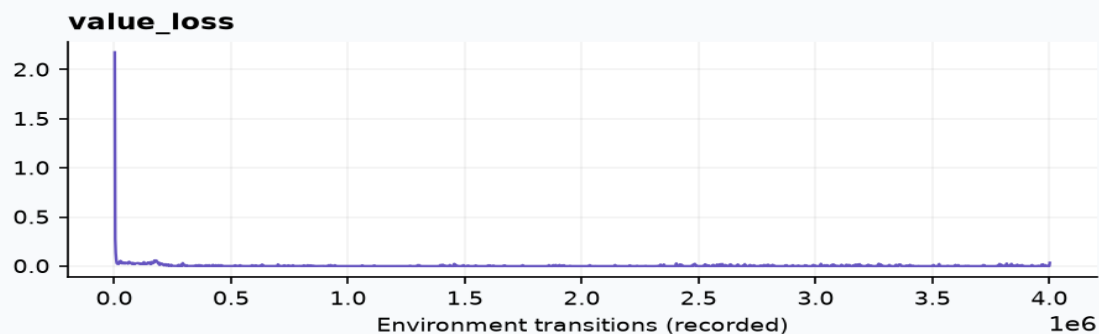
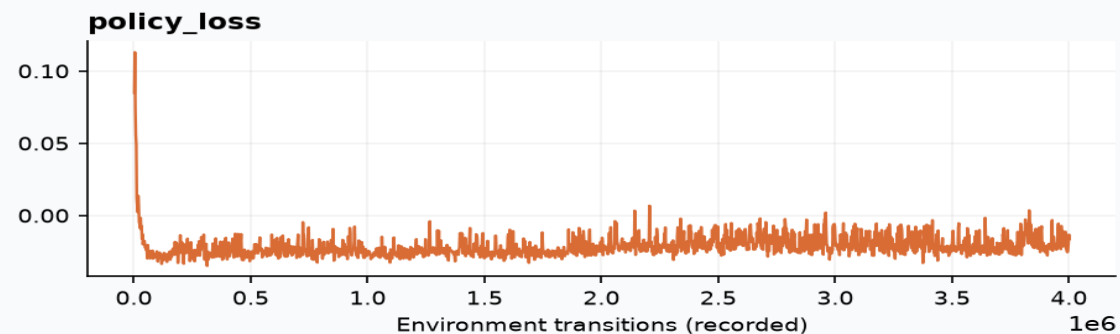
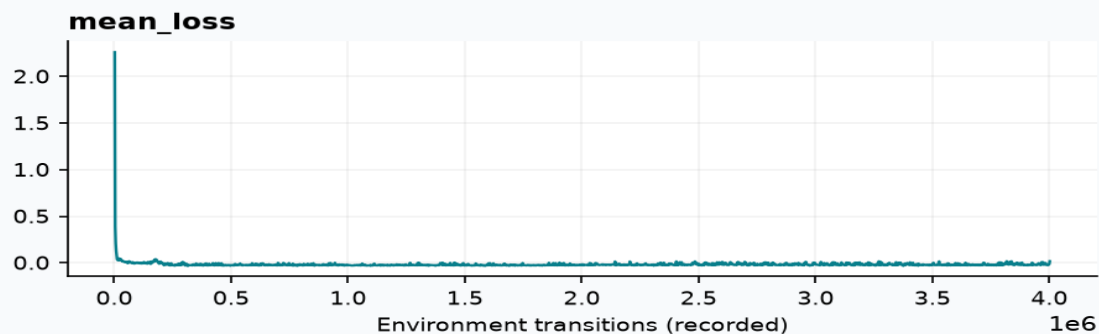
Recorded reward signals — different definitions, not interchangeable



Training reward is an optimization signal, not the acceptance test. See [raw/<case>/](#) for complete recorded fields.

Loss: inspect actor, critic and policy drift

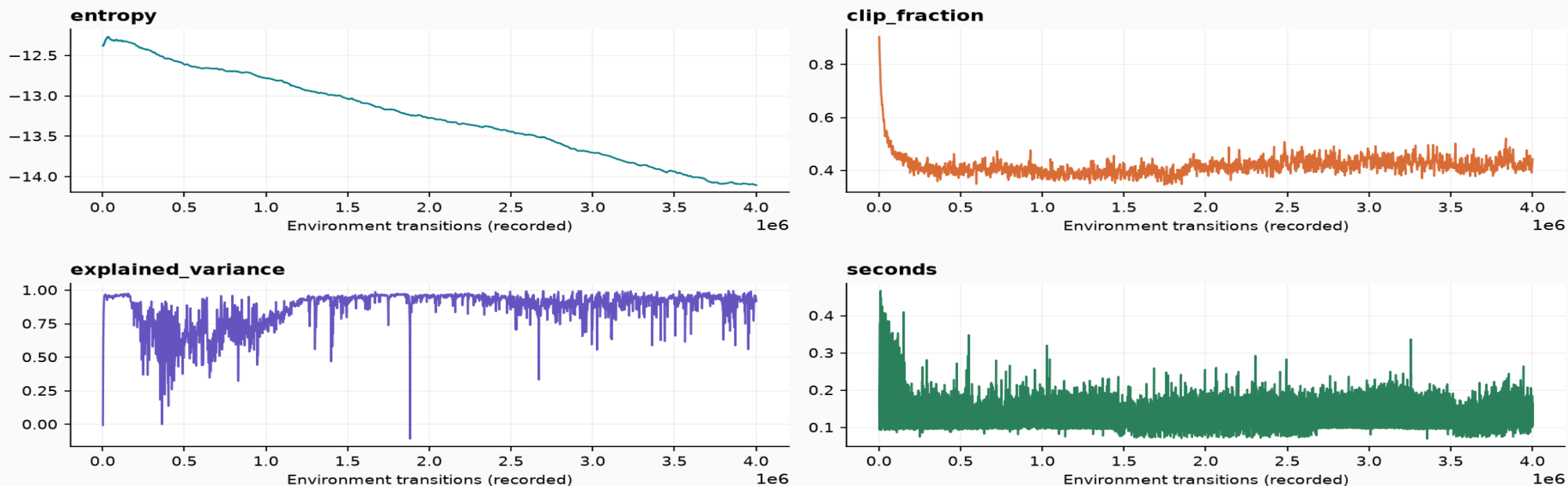
Recorded PPO optimization — raw values, no smoothing



A lower loss does not prove a better robot skill. Loss sign and scale depend on the implementation.

Diagnostics: is the optimizer behaving?

Recorded update diagnostics — absent fields are explicit



Entropy, clipping, explained variance and timing are diagnostics — not a standalone pass/fail certificate.

Metric matrix: numerical audit of this trace · 1/3

- 01 approximate_kl: first 7.759 → final 0.08225; min 0.03034, max 7.759; 1954 logged samples.
- 02 clip_fraction: first 0.9050 → final 0.4437; min 0.3440, max 0.9050; 1954 logged samples.
- 03 entropy: first -12.38 → final -14.10; min -14.10, max -12.27; 1954 logged samples.
- 04 explained_variance: first -0.006817 → final 0.9283; min -0.1084, max 0.9960; 1954 logged samples.

Metric matrix: numerical audit of this trace · 2/3

- 05 mean_loss: first 2.261 → final 0.01790; min -0.03312, max 2.261; 1954 logged samples.
- 06 mean_raw_return: first 1234 → final 5343; min 940.8, max 1.788e+4; 11820 logged samples.
- 07 mean_reward: first 0.2507 → final 0.04188; min 0.03332, max 0.2507; 1954 logged samples.
- 08 policy_loss: first 0.08514 → final -0.01620; min -0.03472, max 0.1133; 1954 logged samples.

Metric matrix: numerical audit of this trace · 3/3

09

value_loss: first 2.176 → final 0.03411; min 0.00004939, max 2.176; 1954 logged samples.

Verification: define success before looking · 1/2

- 01 Require every episode to complete 400 measured steps with no termination, contiguous phase coverage, finite traces, and at least 95% upright samples.
- 02 Require all-joint pose RMSE ≤ 0.15 rad, dynamic gain ≥ 0.20 over a constant-reference-mean baseline, and positive leg dynamic gain.
- 03 Require at least two moving leg joints; the selected reference has 12 moving joints and 8 moving leg joints at the 0.03 rad threshold.
- 04 Compare deterministic trained ONNX against zero action, an untrained network, and the trained network with phase inputs frozen.

Verification: define success before looking · 2/2

05

Validate dynamic batch x 61 input and dynamic batch x 14 output, ONNX checker success, source hashes, and native/ONNX action parity $\leq 1e-4$.

06

Inspect the complete MP4 and captioned contact sheet; reward or finite execution alone cannot establish choreography.

Results: what was actually measured · 1/3

- 01 Actual final evidence: PASS for dance-e2e-20260907-low-noise; Studio evaluation passed 16/16 eight-second episodes with zero terminations.
- 02 Independent seeds 101-105 passed 5/5; mean pose RMSE 0.08438 rad, mean dynamic gain 38.39%, mean leg gain 52.52%, upright coverage 100%.
- 03 Three seeds passed a separate 800-step, 16-second extension that repeats the learned eight-second excerpt twice.
- 04 Zero action passed 0/5 and fell after 43-49 steps; the untrained network fell after 37 steps; frozen phase passed 0/5 for dynamic tracking.

Results: what was actually measured · 2/3

- 05 ONNX/native maximum action error was $3.5762786865234375e-7$; learned tensor L2 change from initialization was 12.247028438523373.
- 06 Training recorded 1,954 finite updates and 31,264 optimizer steps; final-100 policy_loss was -0.01673409083727165.
- 07 Final-100 value_loss was 0.0030093801085240555 and mean_loss was -0.013724710738842986.
- 08 Final-100 approximate_kl was 0.053165218892099804, clip_fraction 0.430621337890625, and explained_variance 0.8950924869797166.

Results: what was actually measured · 3/3

09

Saved render receipt binds ep0.mp4 to ONNX SHA-256
32c27fe239285f1be624b01f77eaa7a411262852c89d51404ae8398ac94bdbd9.

Watch the actual saved rollout



BEFORE PLAY

Predict one success signal and one failure signal.

AFTER PLAY

Which claims does this clip support? What remains untested?

8.0 s saved MP4 • Click in PowerPoint Slide Show • Backup: videos/dance-rollout.mp4

Limits and failure analysis · 1/2

- 01 The claim covers one eight-second excerpt, not the complete 60.923-second source clip or arbitrary choreography.
- 02 Only one successful training seed is recorded; the evaluation seeds do not measure reliability across independently trained policies.
- 03 The recipe changes both exploration scale and sample budget versus earlier failures, so their individual causal effects are not isolated.
- 04 Root translation, root yaw, root roll, and authored contacts are ignored; visible lag, hip-roll offsets, and head-yaw overshoot remain.

Limits and failure analysis · 2/2

05

The run uses nominal XML actuators with no domain randomization, BAM audit, disturbances, terrain, latency, or hardware validation.

06

The 16-second result is two repeats of the same loop, not evidence that unseen source choreography was learned.

Student lab: change one variable, keep the gates · 1/2

- 01 Recommended exercise, not saved evidence: reconstruct all observation slices and identify phase indices 59 and 60.
- 02 Recommended exercise, not saved evidence: recompute dynamic gain from trained-seed101.npz without smoothing, fitted lag, or time shifting.
- 03 Recommended exercise, not saved evidence: calculate rollout count, optimizer count, and sample reuse for a 4,000,000-transition request.
- 04 Recommended exercise, not saved evidence: inspect ONNX names, float32 shapes, and embedded mean, variance, epsilon, and clip constants.

Student lab: change one variable, keep the gates · 2/2

05

Recommended exercise, not saved evidence: design a five-training-seed study with a fixed recipe and predeclared held-out gate.

Submit an experiment, not just a video

- 01** Before training: write one hypothesis, the changed parameter, the fixed baseline, seed list, acceptance criteria and time/transition budget.
- 02** During training: save the exact command, environment versions, initialization, checkpoint and raw reward/loss/KL metrics. Never overwrite the saved reference run.
- 03** After training: evaluate the frozen exported policy on matched seeds; include all failures, aggregate metrics, the full rollout video and a source-bound result table.
- 04** Rubric / 10: reproducibility 2; environment and reward explanation 2; PPO and metrics interpretation 2; honest verification 2; one evidence-based next experiment 2.

Source map and further study · 1/2

01

AGENTS.md

02

microduck_local/AGENTS.md

03

microduck_local/src/microduck_local/contract.py

04

microduck_local/src/microduck_local/walk_env.py

Source map and further study · 2/2

05

`docs/microduck-training-student-book/chapters/02a-ppo-code-lab.md`

06

`docs/microduck-training-student-book/chapters/02b-gae-workshop.md`

07

`scripts/scenarios/README.md`

08

`README.md`